

Lymflow

Masajeador Linfático con ESP32

Embedded Firmware Document

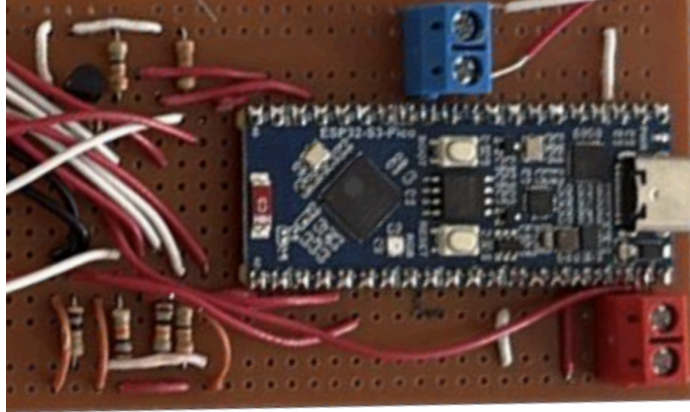
Diseño, arquitectura, firmware y plan de pruebas

Manuela Arboleda, Sebastián Escobar, Laura Gutiérrez, Felipe Vásquez

Sistemas Embebidos

1. Diseños

1.1 Diseño de la PCB



Se utilizó una ESP32 nano para hacer un dispositivo mucho más compacto. Se dejaron borneras para alimentación y motor, en caso de necesitar cambiar el dispositivo.

1.2 Diseño de la carcasa

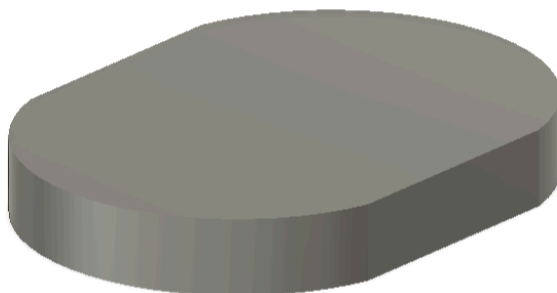
Se partió de un masajeador pre-existente como el que se muestra en la figura, el cuál se le adaptaron una carcasa interna y otra externa para poder tener una correcta lectura de los sensores y contener todos los componentes electrónicos.



Para poder contener el masajeador de manera segura, reposaba sobre un contorno con huecos para que el sistema de presión se mantuviera expuesto al usuario. Para esto es la carcasa inferior que se muestra a continuación:



La carcasa interna permitía tener una superficie rígida para que los sensores de presión tuvieran una superficie con la cuál recibir las deformaciones que recibían de las esferas. Esta se muestra a continuación:



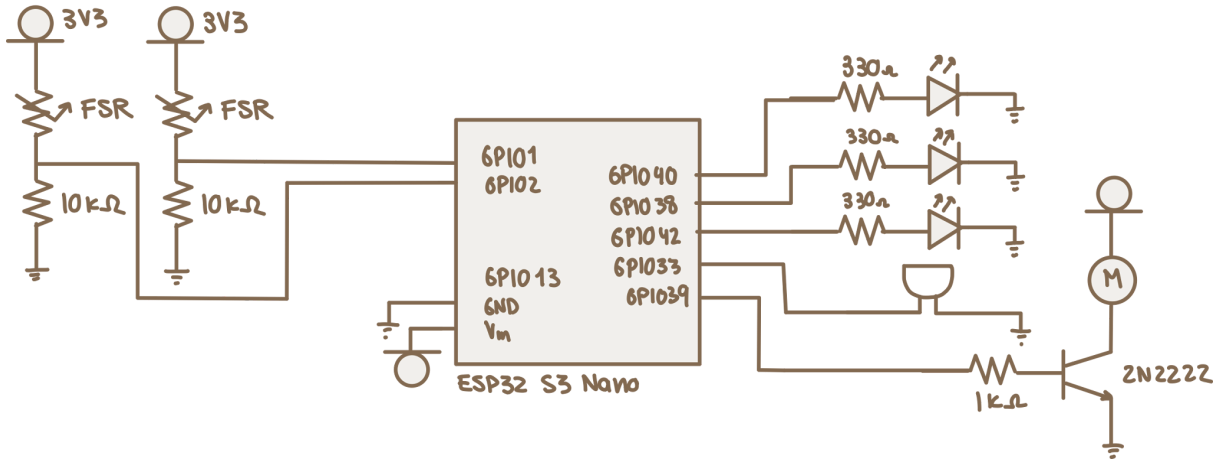
Finalmente, se hizo una cobertura externa a todas las carcasas mencionadas para unificarlas y además tener una estética y ergonomía más adecuada para el usuario.



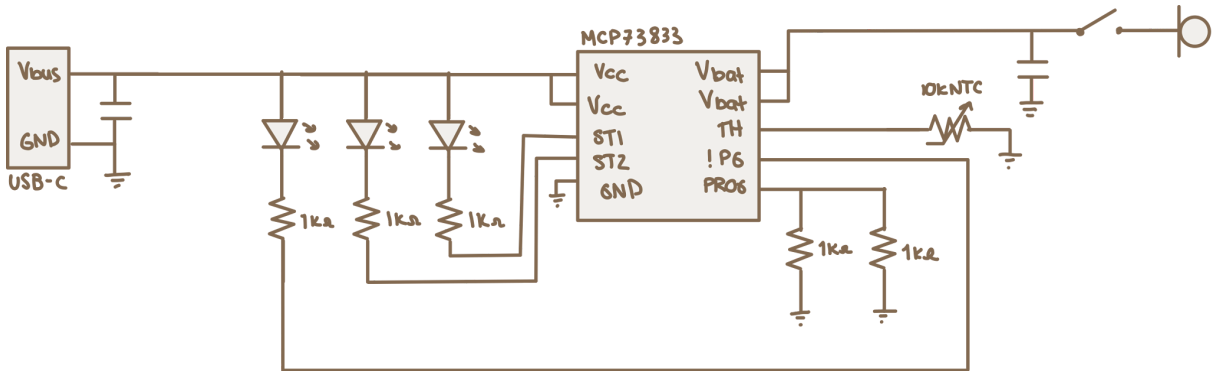
Todas las carcasas fueron impresas en 3D.

2. Esquemáticos

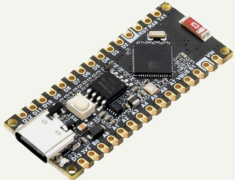
El esquemático principal incluye todas las conexiones de sensado y actuadores hacia el microcontrolador: FSRs para detectar los cambios de la presión ejercida sobre el dispositivo, LEDs para indicar el estado de la presión (insuficiente, óptimo o excesivo), buzzer para indicar presión excesiva, y motor como vibrador para ejercer una presión mucho más apropiada.

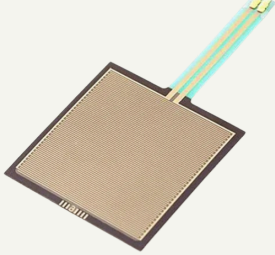


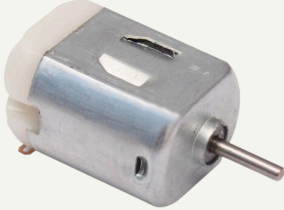
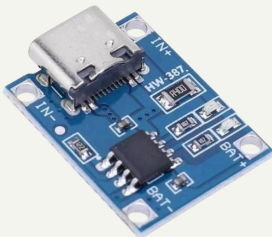
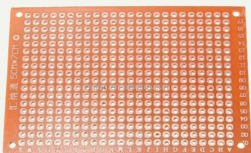
Para la alimentación, se utilizó un cargador USB tipo C para baterías de litio de 3.7v. Incluye este circuito de carga integrado en un solo componente. Es decir, los LEDs y resistencias vienen previamente soldados, a este se le adicionaron un NTC de 10k para protección de temperatura en las baterías, las baterías, un suiche y se conectó directamente a la ESP.



Se pueden resumir los componentes utilizados en la siguiente *Bill of Materials*:

Componente	Imagen	Descripción	Cantidad
ESP32 S3 Nano		Microcontrolador	1

Sensor Resistivo a Presión (FSRs)		Varían su resistencia en función de la presión ejercida	2
Resistencias (10kΩ)		Necesarias para referenciar la señal de las FSRs	2
LEDs		Se utilizó uno amarillo, otro verde y otro rojo para indicar el nivel de presión	3
Resistencias (330Ω)		Necesarias para regular la corriente hacia los LEDs	3
Buzzer		Como alarma en el escenario de presión excesiva	1
Transistor BJT 2N2222		Para aislar la corriente del motor de la corriente del ESP	1

Resistencia (1k Ω)		Para transitar el BJT	1
Motor		Se utilizó como elemento de vibración. Para esto se le adiciona una masa excéntrica	1
Cargador USB-C		Interfaz de carga para llevar la potencia de las baterías al ESP32 y poderlas cargar nuevamente	1
NTC 10k Ω		Para que el cargador se apague si se calientan las baterías	1
Suiche de codo		Para encender el sistema	1
PCB universal (baquelita)		Para soldar los componentes	1

Borneras		Para tener conexiones estables removibles	2
----------	---	---	---

3. Arquitectura de hardware

El sistema Lymflow está basado en un microcontrolador ESP32 (S3) como unidad central de procesamiento. El hardware se organiza en los siguientes subsistemas:

3.1 Subsistema de sensado

Dos sensores de fuerza resistiva (FSR) conectados a los canales ADC1_CH0 y ADC1_CH1. La lectura se realiza con el driver `adc_oneshot` de ESP-IDF con calibración por curva de ajuste lineal (curve fitting). El promedio de ambas lecturas determina el porcentaje de presión aplicada dentro del rango configurado (0–3100 mV).

3.2 Subsistema de actuación

Motor de vibración controlado mediante señal PWM generada con el periférico LEDC del ESP32 en el canal `LEDC_CHANNEL_0` (GPIO 39). La intensidad fija de operación es configurable por medio de la selección de una “rutina de uso” en la interfaz, teniendo 3 opciones de intensidad de vibración posibles.

3.3 Subsistema de retroalimentación

Tres LEDs (verde GPIO 38, amarillo GPIO 40, rojo GPIO 42) y un buzzer (GPIO 33) configurados como salidas digitales. La activación depende de la zona de presión detectada: insuficiente activa LED amarillo, óptima activa LED verde, excesiva activa LED rojo y buzzer.

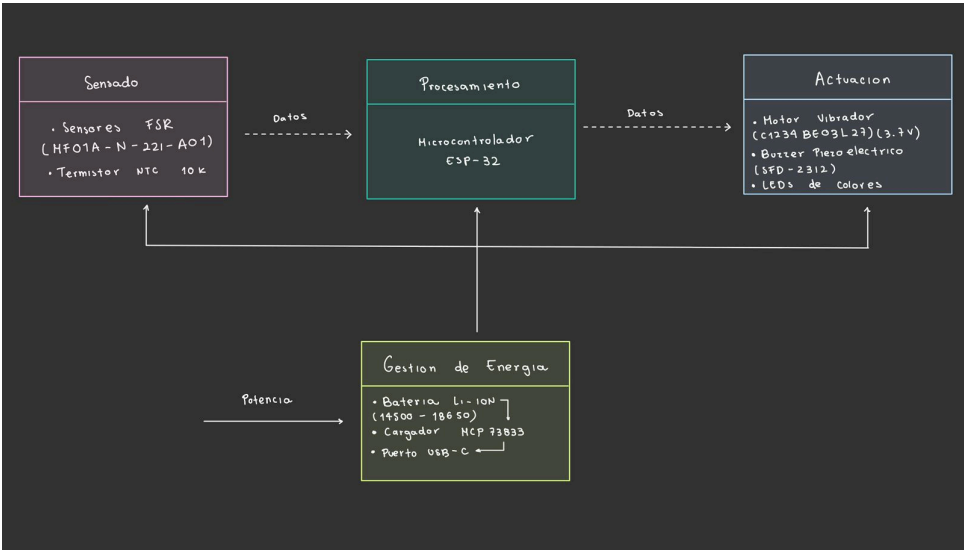
3.4 Subsistema de comunicación

El ESP32 opera en modo Access Point WiFi (sin router externo), generando la red "Masajeador" con autenticación abierta. Un servidor HTTP nativo de ESP-IDF sirve el dashboard web y expone el endpoint `/data` con datos en formato JSON actualizados a 10 Hz.

3.5 Subsistema de alimentación

Por medio de un cable USB-C se alimenta al cargador, encargado de distribuir y cargar las baterías quienes a su vez se encargan de suministrar energía a la ESP32 con sus respectivas cargas. Cuenta con regulación nativa de temperatura e indicación por medio de LEDs integrados.

A continuación se muestra en un diagrama de bloques el flujo de la información dentro de los componentes:



4. Arquitectura de firmware

El firmware está desarrollado sobre el framework ESP-IDF usando C++ con arquitectura orientada a objetos. Cada subsistema de hardware está encapsulado en una clase independiente bajo el directorio lib/, siguiendo el principio de responsabilidad única.

4.1 Estructura de módulos

Módulo	Archivo	Responsabilidad
MANAGER	lib/MANAGER/MANAGER.h/.cpp	Orquestador principal. Inicializa y coordina todos los módulos. Ejecuta el ciclo de control en update() cada 100 ms, incluyendo lectura del sensor, clasificación de zonas, retroalimentación y comunicación web.
FSR	lib/FSR/FSR.h/.cpp	Lectura de sensores de fuerza resistiva (FSR) vía ADC 1 oneshot de ESP-IDF. Calibración por curva de ajuste lineal.
MOTOR	lib/MOTOR/MOTOR.h/.cpp	Control del motor vibrador mediante señal PWM generada con el periférico LEDC del ESP32. Utiliza timer de 10 bits, 2 kHz, y modo Low Speed.
FEEDBACK	lib/FEEDBACK/FEEDBACK.h/.cpp	Sistema de retroalimentación visual y sonora: LEDs verde/amarillo/rojo y buzzer según zona de presión. Fueron configurados como salidas GPIO con resistencias de pull-up.
UART	lib/UART/UART.h/.cpp	Comunicación serial para depuración y monitoreo. Funciona únicamente si se conecta el microcontrolador a un computador. Configurada como 96008N1.
WEBSERVER	lib/WEBSERVER/WEBSERVER.h/.cpp	Access Point WiFi + servidor HTTP nativo ESP-IDF. Sirve landing page, dashboard y página de logs (todo el servidor). Expone los endpoints (handles) de datos, logs y recibe los botones de control.
LOGBUFFER	lib/LOGBUFFER/LOGBUFFER.h/.cpp	Buffer circular estático de hasta 50 entradas de log protegido por mutex FreeRTOS (cierra buffer antes de leer). Serializa a JSON para convertir las estructuras a un string de texto que viaje por el HTTP y el Java Script si pueda entender.

Con el fin de tener una arquitectura mucho más estructurada y limpia, el control principal de todos los subsistemas se realizó en la clase Manager para dejar el main como capa principal, de manera mucho más sencilla.

4.2 Flujo de ejecución principal

El punto de entrada es app_main() en src/main.cpp. La secuencia de inicialización es:

- Inicialización del driver ADC oneshot (adc_oneshot_new_unit)
- Instanciación del objeto Manager con todos los pines configurados
- Llamada a device.setConfig(400, 3100, 0) para definir rango de voltaje y duty cycle del motor
- Llamada a device.init() que inicializa en cascada todos los subsistemas
- Ciclo infinito: device.update() cada 100 ms vía vTaskDelay(pdMS_TO_TICKS(100))

4.3 Ciclo de control (Manager::update)

En cada iteración de 100 ms el Manager ejecuta la siguiente secuencia:

1. Lectura de comandos web: consulta los botones de las 3 figuras del dashboard con getLower/getMiddle/getUpper() y ajusta _motorIntensidad a 100%, 30% y 5%

- respectivamente. También procesa `getStop` para pausar el motor (asignar intensidad 0) o `getResume()` para volver a la velocidad previamente definida.
2. Lectura de FSRs. Lee los voltajes calibrados a menos de que retorne -1 en cuyo caso hay un error en la lectura, registra un WARN en LogBuffer y aborta el ciclo.
 3. A partir de los rangos definidos de presión, calcula el porcentaje de qué tan alejado está de ese rango. Satura en 100% y lanza un WARN si el voltaje supera el máximo definido.
 4. Actualiza el historial de muestras escribiendo el porcentaje actual en un buffer circular de 10 muestras.
 5. Contador de zona óptima: incrementa `_ticksEnZonaOptima` cuando porcentaje está entre 90.9 y 98.5. Además, convierte estos ticks en formato minutos::segundos para enviarlo al servidor.
 6. Clasifica las zonas: según el porcentaje de presión, la clasifica como insuficiente (<90.9%), óptima (90.9-98.5%), excesivo (> 98.5%). Registran en LogBuffer si hay cambio de zona.
 7. Log UART: manda "FSR: X mV | Presion: X%" en serial por cada ciclo para ver en consola las lecturas tanto en voltaje como en porcentaje.
 8. Retroalimentación física: activa `alertasLow()`, `alertasOk()` o `alertasHigh()` según la zona
 9. Calcula desviación estándar de las 10 muestras del historial y define estabilidad alta si `stDev`<5, media si < 15 o baja >= 15.
 10. Actualiza el Web Server por medio de `setData(pct, vDer, minutos, segundos, estabilidad)`.

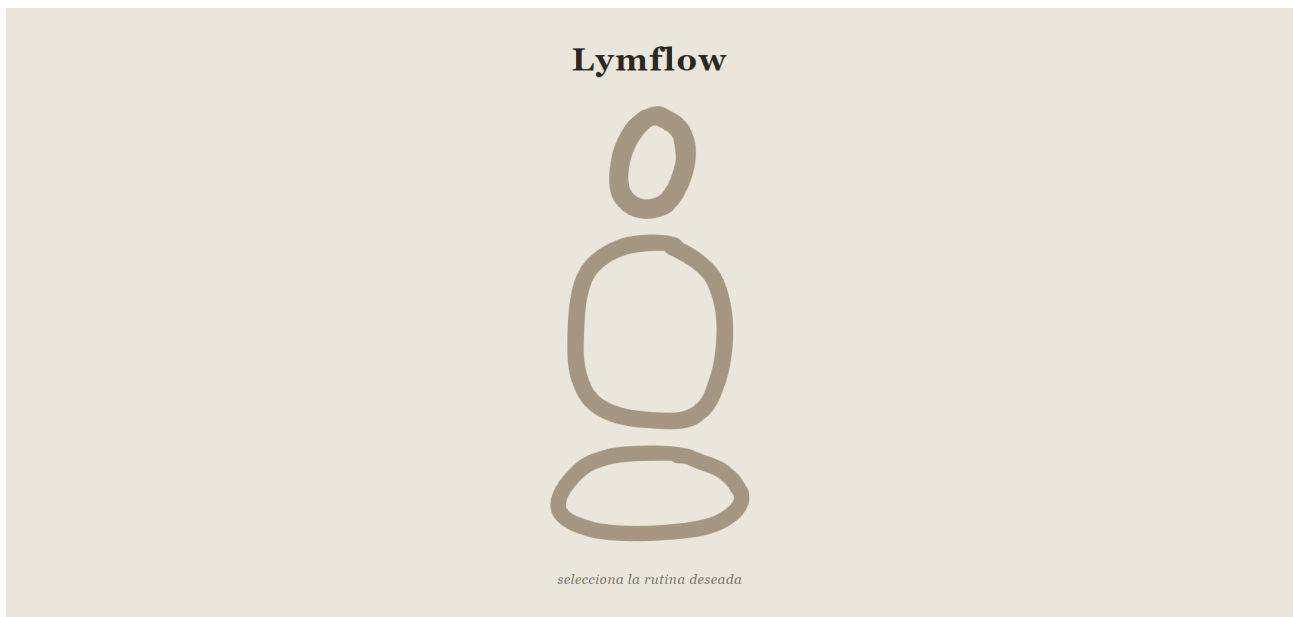
En los Anexos se puede encontrar el diagrama de Estados del documento.

5. Descripción del funcionamiento del sistema

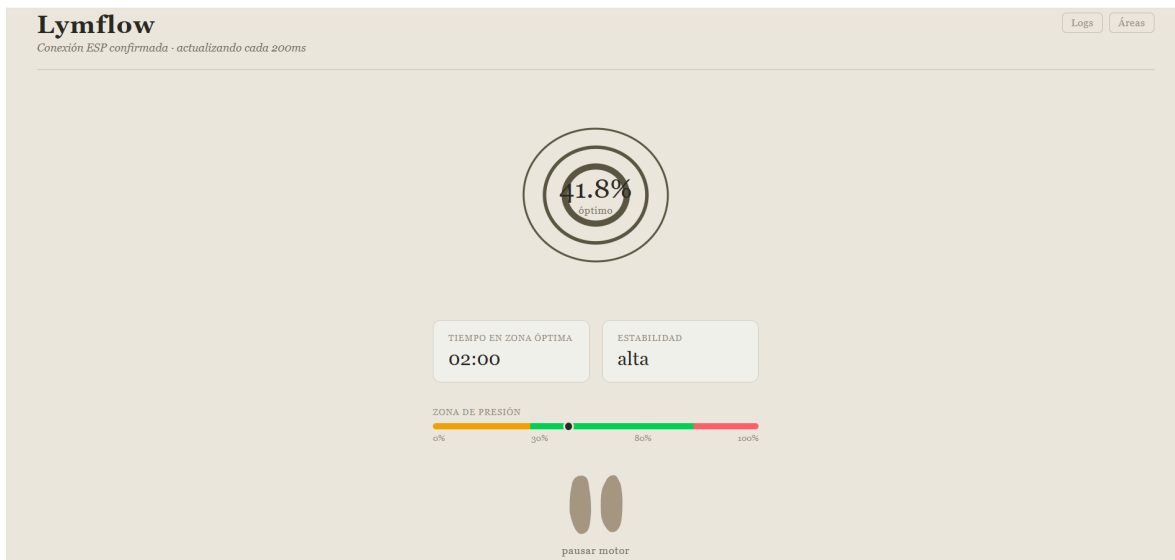
Lymflow es un masajeador linfático inteligente que guía al usuario para aplicar la presión correcta durante una sesión de masaje con el fin de drenar y estimular el sistema linfático, una práctica comúnmente utilizada para el tratamiento de linfedemas o recuperación post-operatoria. El sistema monitorea en tiempo real la fuerza ejercida sobre los sensores FSR y provee retroalimentación inmediata tanto en el dispositivo físico como en una interfaz web accesible desde cualquier smartphone o computador.

5.1 Operación normal

Al encender el dispositivo, el ESP32 inicializa todos los módulos y levanta una red WiFi de nombre "Masajeador". El usuario puede conectarse desde su dispositivo móvil y acceder al dashboard en la IP 192.168.4.1. En el dashboard se muestra una figura en forma de pose de yoga donde cada forma es un botón que permite seleccionar el área a masajear, de manera que el masajeador puede configurar la intensidad de la vibración:



Una vez que selecciona una área, pasa a la interfaz principal donde se leen los datos de los sensores en tiempo real, además de botones interactivos:



A medida que el usuario aplica el dispositivo sobre su piel, los sensores FSR capturan la fuerza ejercida. El firmware calcula el porcentaje de presión y lo clasifica en tres zonas:

- Insuficiente (0–30%): LED amarillo encendido. La presión aplicada es demasiado baja para lograr efecto linfático. El dashboard muestra solo el anillo interior.
- Óptima (30–99.9%): LED verde encendido. La presión está en el rango terapéutico recomendado. El dashboard muestra tres anillos y el contador de tiempo en zona óptima avanza.
- Excesiva (99.9–100%): LED rojo encendido + buzzer activo. La presión es demasiado alta y puede causar malestar. El dashboard muestra todos los anillos.

5.2 Interfaz web

El dashboard web se actualiza automáticamente cada 200 ms mediante polling al endpoint `/data`. Muestra el porcentaje de presión con anillos concéntricos SVG, el tiempo acumulado en zona óptima en formato MM:SS, y la estabilidad de la presión (alta / media / baja) calculada como la desviación estándar de las últimas 10 muestras. Con el fin de mantener el dispositivo como inalámbrico, se añadió también una sección del servidor para

visualizar los LOGS:

Lymflow

Registro del sistema · actualizando cada segundo

WARNINGS

Sin entradas...

ERRORS

Sin entradas...

INFO

T=00:00:10.763

Cliente conectado: cc:47:40:70:6f:96 AID=1

T=00:00:02.488

Estabilidad: media -> alta (stdDev=0.14)

T=00:00:02.389

Estabilidad: baja -> media (stdDev=12.49)

T=00:00:01.688

Estabilidad: media -> baja (stdDev=16.72)

T=00:00:01.593

Estabilidad inicial: media (stdDev=12.57)

T=00:00:01.541

Zona inicial: optimo (41.9%)

T=00:00:01.502

HTTP server iniciado en puerto 80

T=00:00:01.459

AP iniciado – SSID: Masajeador, canal: 1

5.3 Condiciones de alerta

El buzzer se activa únicamente cuando la presión supera el umbral del 80%, indicando al usuario que debe reducir la fuerza aplicada. No hay alertas sonoras en zona insuficiente para no interrumpir la experiencia del masaje, solo retroalimentación visual.

6. Descripción del firmware

6.1 Entorno de desarrollo

- Framework: ESP-IDF vía PlatformIO
- Board: ESP32 S3
- Lenguaje: C++ con orientación a objetos
- IDE: Visual Studio Code + extensión PlatformIO
- Control de versiones: Git a través de GitHub

6.2 Clase FSR

Encapsula la lectura de un sensor FSR sobre un canal ADC del ESP32. Recibe el GPIO y el handle ADC compartido verificando que sea un pin adecuado (si no, entrega un log de ERROR). En `init()` configura el canal con atenuación 12 dB (rango 0–3.3 V) y crea el esquema de calibración por curve. `read()` devuelve el voltaje calibrado en mV; retorna -1 en caso de error de lectura.

6.3 Clase Motor

Controla el motor vibrador mediante el periférico LEDC. `init()` configura el timer 0 en modo low speed con 10 bits de resolución y frecuencia de 5kHz. `setIntensity(uint8_t porcentaje)` traduce el porcentaje (0–100) al duty cycle correspondiente para el timer de 10 bits (*maxduty* 1023), y finalmente se actualiza el periférico con `ledc_ser_duty` y `ledc_update_duty`. El destructor detienen el canal con `ledc_stop`. .

6.4 Clase FeedbackSystem

Gestiona los tres LEDs y el buzzer como salidas GPIO. `init()` los configura a los 4 como modo output con pull-up activo mediante `gpio_config`. Los métodos `alertasLow()`, `alertasOk()` y `alertasHigh()` activan el LED correspondiente y apagan los demás. `alertasHigh()` adicionalmente activa el buzzer.

6.5 Clase Uart

Construída sobre el driver UART de ESP-IDF. Como se mencionó previamente, configurado como 96008N1. `sendUart(const char*)` escribe el mensaje al buffer UART. Se usa para transmitir el log de lecturas en cada ciclo de control.

6.6 Clase WebServer

Inicializa el Access Point WiFi usando `esp_wifi.h` (SSID “Masajeador”, canal 1, autenticación abierta y máximo 4 clientes) y levanta un servidor HTTP con `esp_http_server.h`. Los manejadores estáticos `_handleIndex()` y `_handleData()` responden a GET / y GET /data respectivamente. `setData()` actualiza las variables estáticas compartidas con los valores calculados por el Manager. El JSON de respuesta incluye: pct, vProm, estado, minutos, segundos y estabilidad. Los endpoints registrados son:

URI	Descripción
GET /	Redirige 302 a /landing
GET /landing	Sirve la landing page (selección de rutina de masaje)
GET /dashboard	Sirve el dashboard de monitoreo en tiempo real
GET /logs	Sirve la página de visualización de logs
GET /data	JSON con pct, vProm, estado, minutos, segundos, estabilidad
GET /logs-data	JSON array con hasta 50 entradas de log, más recientes primero
GET /control	Recibe comandos vía query string: stop, resume, Lower, Middle, Upper

6.7 Clase Manager

Es la clase de más alto nivel antes de pasar al main. Ya fue previamente descrita en Arquitectura de Firmware.

6.7 Clase Log Buffer

El estado se controla con dos variables: `_head`, que apunta a la entrada más antigua, y `_count`, que lleva el conteo actual. Cuando el buffer está lleno, cada nueva entrada sobrescribe la más antigua y `_head` avanza. El acceso está configurado con timeouts de 20 ms para escritura y 100 ms para lectura. El timestamp se obtiene con `esp_timer_get_time()` convertido a milisegundos. `fillJson()` recorre el buffer de más reciente a más antiguo, construyendo el JSON carácter a carácter con `snprintf` y escapando manualmente comillas y backslashes para garantizar JSON válido a la hora de serializar hacia el servidor. Se pueden resumir los Logs a continuación:

Archivo Fuente	Nivel	Mensaje (Message)	Descripción
FSR.cpp	ERROR	Pin %d no es un canal ADC1 valido (usar 32/33/34/35/36/39)	Si se seleccionó un pin que no soporta ADC
	ERROR	FSR pin %d: init omitido, canal invalido	Error en la definición del pin del FSR
	WARN	ADC calibración no disponible en pin %d, usando valores crudos	Si falla la calibración
MANAGER.cpp	WARN	FSR izq fallo en lectura ADC, omitiendo ciclo	Para no mostrar lecturas fallidas
	WARN	FSR der fallo en lectura ADC, omitiendo ciclo	Para no mostrar lecturas fallidas
	WARN	Presión saturada: vPromedio=%d vMax=%d	Si se sale de los rangos establecidos
	INFO	Zona inicial: %s (%.1f%%)	Zona donde inició
	INFO	Zona: %s -> %s (%.1f%%)	Informa cambio de zona
	INFO	Estabilidad inicial: %s (stdDev=%.2f)	Estabilidad al inicio
	INFO	Estabilidad: %s -> %s (stdDev=%.2f)	Estabilidad en un momento determinado
WEBSERVER.cpp (WiFi init)	INFO	Cliente conectado: XX:XX:XX:XX:XX:XX AID=%d	Conexión exitosa con un cliente
	INFO	Cliente desconectado: XX:XX:XX:XX:XX:XX AID=%d razon=%d	Desconexión con cliente
	WARN	NVS corrupta, borrando y re-inicializando	Si no se pudo guardar en el Non-Volatile Storage que es un área de la Flash
	ERROR	NVS init fallo tras borrado: %d	Si falla inicialización después del borrado
	ERROR	Event loop create fallo: %d	No se pudo crear el bus interno de eventos
	ERROR	esp_wifi_init fallo: %d	No se inicializó el driver de WiFi
	ERROR	esp_wifi_set_mode fallo: %d	No se pudo configurar el ESP32 en modo Access Point (Ser router, no conectarse a uno - station)
	ERROR	esp_wifi_set_config fallo: %d	No se pudo aplicar la configuración de la red (SSID, canal, autenticación)
	ERROR	esp_wifi_start fallo: %d	El WiFi se configuró bien pero no se pudo encender
	INFO	AP iniciado — SSID: Masajeador, canal: 1	Todo OK, red activa
	ERROR	HTTP server no pudo iniciar: %d	No pudo iniciar el servidor
WEBSERVER.cpp (HTTP server)	INFO	HTTP server iniciado en puerto %d	Se inició el servidor, está corriendo y esperando peticiones
	ERROR	Registro /data falló	El servidor arrancó pero no pudo activar la ruta /data, esa URL nunca responderá peticiones
	ERROR	Registro / falló	El servidor arrancó pero no pudo activar la ruta / (defecto), esa URL nunca responderá peticiones
	ERROR	Registro /logs falló	El servidor arrancó pero no pudo activar la ruta /logs, esa URL nunca responderá peticiones
	ERROR	Registro /logs-data fallo	El servidor arrancó pero no pudo activar la ruta /logs-data, esa URL nunca responderá peticiones
	ERROR	Registro /control fallo	El servidor arrancó pero no pudo activar la ruta /control, esa URL nunca responderá peticiones
	ERROR	Registro /landing fallo	El servidor arrancó pero no pudo activar la ruta /landing, esa URL nunca responderá peticiones
	ERROR	Registro /dashboard fallo	El servidor arrancó pero no pudo activar la ruta /dashboard, esa URL nunca responderá peticiones
	WARN	/data envio fallo: %d	Si falla el registro y la ruta queda inactiva. El servidor se queda corriendo pero la URL no responde a nadie. Normalmente falla por falta de memoria o tope de

			handlers.
	WARN	/dashboard envio fallo: %d	Si falla el registro y la ruta queda inactiva. El servidor se queda corriendo pero la URL no responde a nadie.
	WARN	/logs envio fallo: %d	Si falla el registro y la ruta queda inactiva. El servidor se queda corriendo pero la URL no responde a nadie.
	ERROR	/logs-data sin memoria heap	Si se llena el heap y no se puede construir el string de los logs
	WARN	/logs-data envio fallo: %d	Si falla el registro y la ruta queda inactiva. El servidor se queda corriendo pero la URL no responde a nadie.
	WARN	/landing envio fallo: %d	Si falla el registro y la ruta queda inactiva. El servidor se queda corriendo pero la URL no responde a nadie.

7. Matriz de trazabilidad (SRTM)

La siguiente matriz relaciona cada requerimiento funcional y no funcional con su test case asociado, el tipo de prueba y el estado de ejecución.

Req ID	Requerimiento	Test Case ID	Tipo	Estado	Artefactos
RF-01	Vibración variable 5–30 Hz	TC-002	Unitario	Pendiente	Señal PWM / serial plotter
RF-02	Presión controlada 15–40 mmHg	TC-006, TC-007, TC-008	Funcional	Pendiente	Serial plotter + interfaz web
RF-03	Temperatura dispositivo < 40°C	TC-010	Usuario / Seg.	Pendiente	Termómetro infrarrojo
RF-04	Autonomía ~9 horas (6 sesiones)	TC-011	Usuario	Pendiente	Registro descarga batería
RF-05	Tolerancia < 2 mm en elementos	TC-009	Usuario	Pendiente	Inspección física / medición
RF-06	RSSI > -70 dBm	TC-004	Integración	Pendiente	Monitor serial / herramienta WiFi
RF-07	Pérdida de paquetes < 1.2%	TC-004	Integración	Pendiente	Logs de comunicación
RF-08	Protección IP65	TC-012	Sistema	Pendiente	Prueba polvo/agua (video/fotos)
RF-09	Peso < 500 g	TC-009	Usuario	Pendiente	Gramera
RF-10	Ruido < 50 dB	TC-013	Sistema	Pendiente	Sonómetro / app dB
RF-11	Actualización ≥ 1 Hz en interfaz	TC-004	Integración	Pendiente	Captura interfaz web
RNF-01	Respuesta < 2 s en interfaz web	TC-004	Integración	Pendiente	Captura + cronómetro

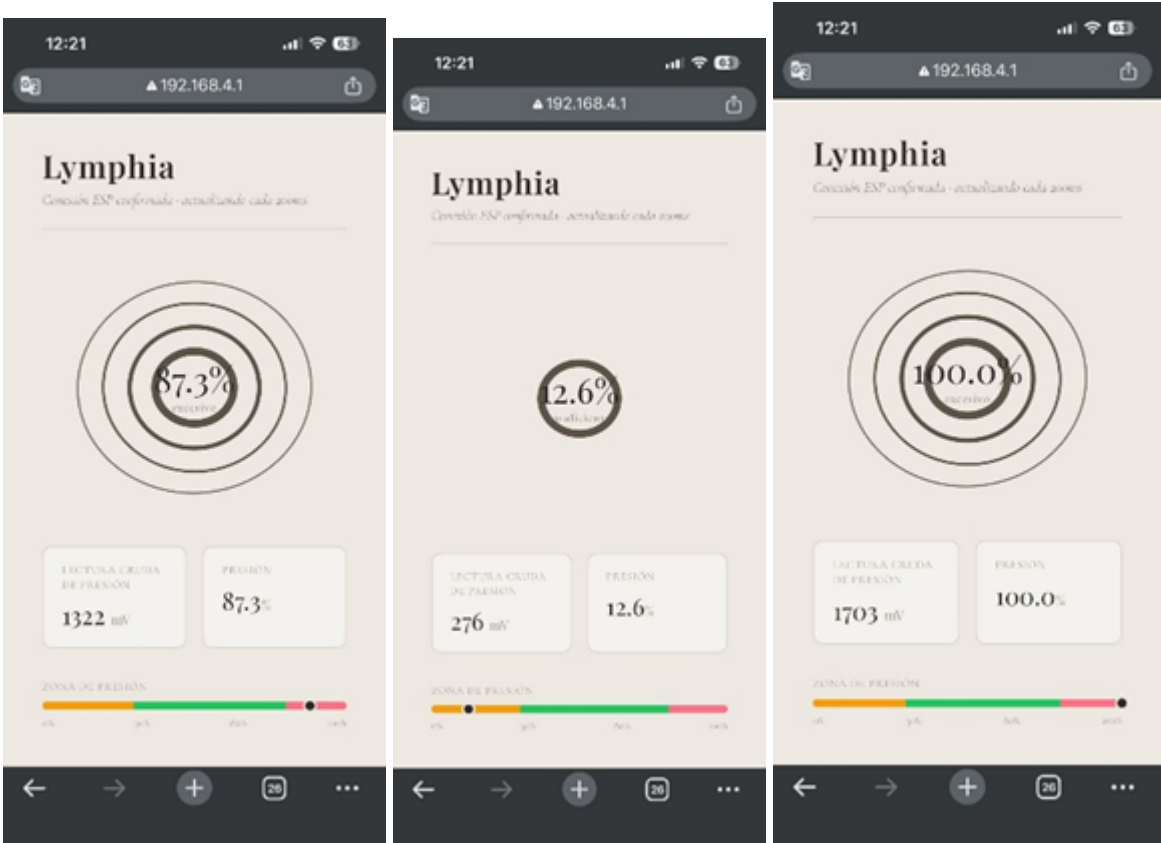
RNF-02	Facilidad de uso	TC-009	Usuario	Pendiente	Encuesta / observación
RNF-03	Firmware modular	TC-001, TC-002	Unitario	Pendiente	Revisión de código
RNF-04	Código documentado	TC-001	Unitario	Pendiente	Revisión del repositorio
RNF-05	Limpieza y desinfección	TC-010	Usuario	Pendiente	Evidencia física
RNF-06	Materiales hipoalergénicos	TC-009	Usuario	Pendiente	Ficha técnica materiales

8. Anexo — Test Cases

A continuación se presentan los 13 test cases del plan de pruebas organizados por tipo. Los campos de Resultado Real, Evidencia y Status General deben completarse durante la ejecución de las pruebas.

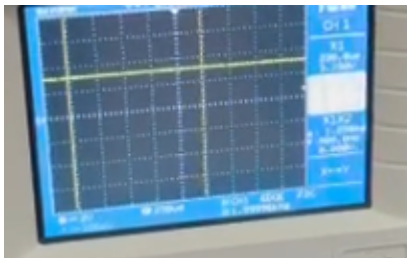
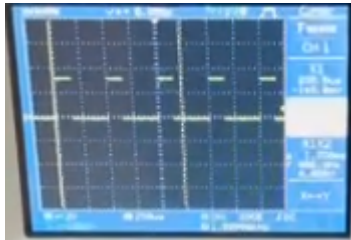
8.1 Pruebas unitarias

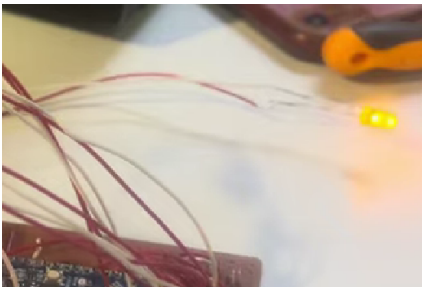
TC-001 Sensor de Presión Unitario			
Requerimiento	RF-02 – El sensor de presión debe entregar señal estable dentro del rango de operación		
Objetivo	Verificar que el sensor de presión entregue una señal estable y coherente dentro del rango de trabajo		
Precondiciones	ESP32 encendido, sensor de presión conectado, serial plotter abierto en PlatformIO		
PASOS DEL TEST			
#	INPUT	RESULTADO ESPERADO	STATUS
1	Encender el sistema sin aplicar presión al sensor	Valor de presión ≈ 0 (reposo), señal estable sin fluctuaciones mayores a ± 2 unidades	<i>[X] Pasa [] Fail</i>
2	Aplicar presión gradual dentro del rango nominal (15–40 cmH ₂ O)	Señal sube de forma proporcional y suave, sin saltos bruscos ni ruido excesivo	<i>[] Pasa [X] Fail</i>
3	Mantener presión constante durante 10 s y observar el plotter	Valor se estabiliza; variación $\leq \pm 2$ unidades durante la ventana de observación	<i>[X] Pasa [] Fail</i>
VALIDACIÓN			
Resultado Real	El prototipo tiene una ESP que no maneja un suficiente overlapping, y por esto el #2 no se ejecuta de forma adecuada		
EVIDENCIA			



STATUS GENERAL:	[X] PASÓ [] FALLÓ [] SE BLOQUEÓ [] NO SE EJECUTÓ
-----------------	---

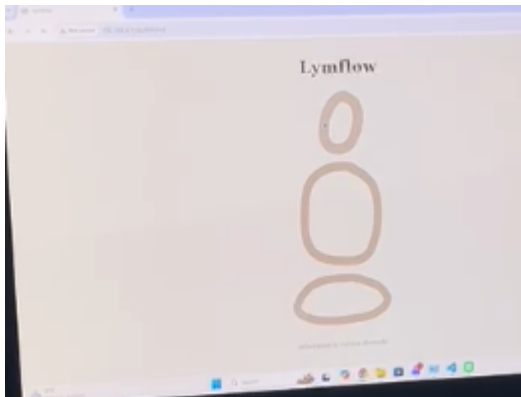
TC-002 Motor / PWM Unitario			
Requerimiento	RF-01 – Vibración variable entre 5–30 Hz mediante señal PWM		
Objetivo	Validar que el motor vibrador sea activado correctamente mediante señal PWM generada desde el ESP32		
Precondiciones	ESP32 encendido, motor conectado al driver correspondiente, osciloscopio o serial plotter disponible		
PASOS DEL TEST			
#	INPUT	RESULTADO ESPERADO	STATUS
1	Configurar duty cycle al 20% desde firmware (≈ 5 Hz)	Motor vibra a baja intensidad; señal PWM visible en osciloscopio	<i>[X] Pasa [] Fail</i>
2	Aumentar duty cycle al 60% (frecuencia media)	Intensidad de vibración aumenta perceptiblemente; señal PWM aumenta proporcionalmente	<i>[X] Pasa [] Fail</i>

3	Llevar duty cycle al 100% (≈ 30 Hz / máxima intensidad)	Motor vibra a máxima intensidad; no se detectan oscilaciones anormales	[X] Pasa[] Fail
VALIDACIÓN			
Resultado Real	Cumple esperadamente las expectativa		
EVIDENCIA			
			
STATUS GENERAL:	[X] PASÓ [] FALLÓ [] SE BLOQUEÓ [] NO SE EJECUTÓ		

TC-003 Indicador LED Unitario			
Requerimiento	RF-05 – Indicador LED se enciende al encender el sistema		
Objetivo	Confirmar que el indicador LED se enciende correctamente al iniciar el sistema		
Precondiciones	ESP32 con firmware cargado, LED conectado al GPIO correspondiente, sistema apagado		
PASOS DEL TEST			
#	INPUT	RESULTADO ESPERADO	STATUS
1	Conectar la alimentación al ESP32	El LED indicador se enciende en menos de 2 s tras aplicar alimentación	[X] Pasa[] Fail
2	Verificar visualmente el estado del LED	LED encendido de forma continua y estable, sin parpadeo no intencional	[X] Pasa[] Fail
3	Desconectar la alimentación	LED se apaga inmediatamente al retirar la alimentación	[X] Pasa[] Fail
VALIDACIÓN			
Resultado Real	Cumple esperadamente las expectativas		
EVIDENCIA			
			

STATUS GENERAL:	☒ PASÓ ☐ FALLÓ ☐ SE BLOQUEÓ ☐ NO SE EJECUTÓ
------------------------	--

8.2 Pruebas de integración

TC-004 Comunicación ESP32 ↔ Web Integración			
Requerimiento	RF-06 / RF-07 / RF-11 / RNF-01 – Datos enviados por el ESP32 son recibidos y procesados por la interfaz web		
Objetivo	Verificar que los datos enviados por el ESP32 sean recibidos y procesados correctamente por la interfaz web		
Precondiciones	ESP32 y dispositivo cliente en la misma red Wi-Fi; servidor web activo; navegador abierto en la IP del ESP32		
PASOS DEL TEST			
#	INPUT	RESULTADO ESPERADO	STATUS
1	Conectar el navegador web a la dirección IP del ESP32	La interfaz web carga correctamente y muestra el panel de monitoreo	<i>[X] Pasa [] Fail</i>
2	Aplicar presión al sensor; observar valores en la web	Los valores de presión mostrados corresponden en tiempo real a los datos del sensor (latencia ≤ 500 ms)	<i>[X] Pasa [] Fail</i>
3	Cambiar parámetros de vibración desde la interfaz web	El ESP32 recibe el comando y actualiza la señal PWM; el cambio se confirma visualmente en la UI	<i>[X] Pasa [] Fail</i>
VALIDACIÓN			
Resultado Real	Cumple satisfactoriamente las expectativas		
EVIDENCIA			
			
STATUS GENERAL:	[X] PASÓ [] FALLÓ [] SE BLOQUEÓ [] NO SE EJECUTÓ		

TC-005 Servidor Web / Red Integración	
Requerimiento	RF-06 – El sistema debe detectar desconexión del servidor
Objetivo	Validar que el sistema detecta y reporta correctamente la desconexión del cliente web

Precondiciones	ESP32 encendido con servidor activo, cliente conectado desde el navegador		
PASOS DEL TEST			
#	INPUT	RESULTADO ESPERADO	STATUS
1	Con el cliente conectado, cerrar abruptamente el navegador	El ESP32 detecta la desconexión en ≤ 5 s y registra el evento	<i>[X] Pasa [] Fail</i>
2	Desconectar el cliente de la red Wi-Fi sin cerrar el navegador	El servidor detecta timeout de conexión y libera el socket correctamente	<i>[X] Pasa [] Fail</i>
3	Volver a conectar el cliente	La reconexión se establece exitosamente sin reiniciar el ESP32	<i>[X] Pasa [] Fail</i>
VALIDACIÓN			
Resultado Real	Cumple satisfactoriamente las expectativas		
EVIDENCIA			
			
STATUS GENERAL:	[X] PASÓ [] FALLÓ [] SE BLOQUEÓ [] NO SE EJECUTÓ		

8.3 Pruebas de sistema (funcionales)

TC-006 Sensor / Calibración Sistema (Funcional)	
Requerimiento	RF-02 – Verificar offset: presión registrada = 0 cuando el sensor está en reposo
Objetivo	Verificar que el sistema registra 0 de presión cuando el sensor está en reposo
Precondiciones	ESP32 encendido, sensor sin ninguna presión aplicada, serial plotter o interfaz web activa

PASOS DEL TEST			
#	INPUT	RESULTADO ESPERADO	STATUS
1	Iniciar el sistema con el sensor libre (sin presión)	Lectura de presión muestra $0 \pm$ banda de ruido aceptada (≤ 2 unidades)	<i>[X] Pass [] Fail</i>
2	Dejar el sistema en reposo 60 s y registrar valores	Los valores permanecen estables; no hay drift significativo	<i>[X] Pass [] Fail</i>
3	Mover ligeramente el cable del sensor sin aplicar presión real	Cualquier variación es transitoria; lectura vuelve a zona de reposo en < 3 s	<i>[X] Pass [] Fail</i>
VALIDACIÓN			
Resultado Real	Los valores no son completamente estables, pero tampoco son significativos, la lectura de presión cuando el dispositivo esta en reposos es entre 260mv-270		
EVIDENCIA			
			
STATUS GENERAL:	[X] PASÓ [] FALLÓ [] SE BLOQUEÓ [] NO SE EJECUTÓ		

TC-007 Interfaz Web / Presión Sistema (Funcional)	
Requerimiento	RF-02 – Cambios visuales ante variaciones de presión por encima del umbral
Objetivo	Observar que la interfaz web muestra cambios visuales cuando la presión supera el umbral definido
Precondiciones	Sistema en línea, interfaz web activa, umbral superior configurado en firmware

PASOS DEL TEST			
#	INPUT	RESULTADO ESPERADO	STATUS
1	Aplicar presión gradualmente hasta justo por debajo del umbral superior	Interfaz muestra indicador en zona normal (color verde o neutro)	<i>[X] Pasa [] Fail</i>
2	Superar el umbral superior de presión	Interfaz cambia a indicación de alerta (color rojo, barra al límite)	<i>[X] Pasa [] Fail</i>
3	Reducir presión por debajo del umbral	La indicación de alerta desaparece y la interfaz regresa al estado normal	<i>[X] Pasa [] Fail</i>
VALIDACIÓN			
Resultado Real	Cumple satisfactoriamente las expectativas		
EVIDENCIA			
Para evidencia física se necesita presenciar en video o en tiempo real			
STATUS GENERAL:	[X] PASÓ [] FALLÓ [] SE BLOQUEÓ [] NO SE EJECUTÓ		

TC-008 Interfaz Web / Presión Sistema (Funcional)			
Requerimiento	RF-02 – Cambios visuales ante variaciones de presión por debajo del umbral		
Objetivo	Observar que la interfaz web muestra cambios visuales cuando la presión cae por debajo del umbral mínimo		
Precondiciones	Sistema en línea, interfaz web activa, umbral inferior configurado en firmware		
PASOS DEL TEST			
#	INPUT	RESULTADO ESPERADO	STATUS
1	Operar el dispositivo dentro del rango nominal de presión	Interfaz muestra estado normal; indicadores en zona verde/aceptable	<i>[X] Pasa [] Fail</i>
2	Reducir la presión hasta quedar por debajo del umbral mínimo	Interfaz muestra indicación de presión insuficiente (color amarillo o barra en zona baja)	<i>[X] Pasa [] Fail</i>
3	Volver a aplicar presión dentro del rango nominal	La indicación de presión baja desaparece; interfaz regresa a estado normal	<i>[X] Pasa [] Fail</i>
VALIDACIÓN			
Resultado Real	Cumple satisfactoriamente las expectativas		
EVIDENCIA			
Para evidencia física se necesita presenciar en video o en tiempo real			

STATUS GENERAL:	☒ PASÓ ☐ FALLÓ ☐ SE BLOQUEÓ ☐ NO SE EJECUTÓ
------------------------	--

TC-012 Hardware / Protección Sistema (Funcional)			
Requerimiento	RF-08 – El dispositivo resiste exposición a polvo y agua sin afectar funcionamiento interno		
Objetivo	Verificar funcionamiento normal tras exposición controlada a polvo y salpicaduras de agua		
Precondiciones	Prototipo ensamblado con carcasa, paño húmedo y entorno con polvo disponible		
PASOS DEL TEST			
#	INPUT	RESULTADO ESPERADO	STATUS
1	Exponer la superficie externa a polvo fino durante 30 s y limpiar con paño seco	El polvo no penetra al interior; el dispositivo opera normalmente tras la limpieza	<i>[X] Pasa [] Fail</i>
2	Aplicar salpicaduras de agua en la carcasa (no sumersión) desde 30 cm	El agua no ingresa al interior; el dispositivo continúa operando sin interrupciones	<i>[X] Pasa [] Fail</i>
3	Operar el dispositivo durante 2 min post-exposición	Todas las funciones (sensor, motor, LED, Wi-Fi) responden correctamente	<i>[X] Pasa [] Fail</i>
VALIDACIÓN			
Resultado Real	Cumple satisfactoriamente las expectativas		
EVIDENCIA			
			
STATUS GENERAL:	[X] PASÓ [] FALLÓ [] SE BLOQUEÓ [] NO SE EJECUTÓ		

TC-013 Hardware / Acústica Sistema (Funcional)	
Requerimiento	RF-10 – Nivel de ruido del dispositivo no supera los 50 dB durante operación
Objetivo	Medir el nivel de ruido del dispositivo en operación
Precondiciones	Aplicación medidora de dB instalada en smartphone (o sonómetro), entorno silencioso (ruido ambiente < 35 dB)
PASOS DEL TEST	

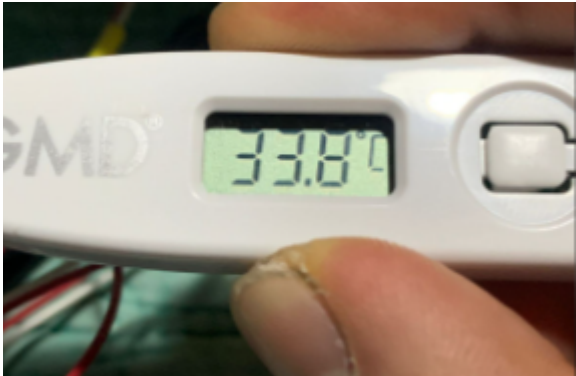
#	INPUT	RESULTADO ESPERADO	STATUS
1	Medir ruido ambiente sin encender el dispositivo	Ruido de fondo < 35 dB; se establece línea base	[X] Pasa[] Fail
2	Encender el dispositivo en modo nominal (motor al 60%) y medir a 30 cm	Nivel de ruido ≤ 50 dB a 30 cm del dispositivo	[X] Pasa[] Fail
3	Subir motor al 100% y volver a medir	Incluso a máxima intensidad, el nivel de ruido no supera los 50 dB a 30 cm	[X] Pasa[] Fail
VALIDACIÓN			
Resultado Real	Cumple satisfactoriamente las expectativas		
EVIDENCIA			
			
STATUS GENERAL:	[X] PASÓ [] FALLÓ [] SE BLOQUEÓ [] NO SE EJECUTÓ		

8.4 Pruebas de usuario y seguridad

TC-009 Hardware / Ergonomía Usuario y Seguridad			
Requerimiento	RNF-02 / RF-09 – Ergonomía evaluada mediante pruebas de agarre manual		
Objetivo	Evaluar la ergonomía del dispositivo con al menos 3 usuarios distintos		
Precondiciones	Prototipo físico disponible, al menos 3 evaluadores, formulario de evaluación ergonómica preparado		
PASOS DEL TEST			
#	INPUT	RESULTADO ESPERADO	STATUS

1	Solicitar a cada evaluador que tome el dispositivo con la mano dominante durante 2 min	El 80% de los evaluadores reporta agarre cómodo sin esfuerzo excesivo	[X] Pasa[] Fail
2	Solicitar al evaluador que opere los controles sin ayuda	El evaluador puede activar/desactivar el dispositivo sin cambiar el agarre	[X] Pasa[x] Fail
3	Evaluar fatiga muscular tras 5 min de uso continuo	Ningún evaluador reporta dolor; calificación promedio de comodidad ≥ 3/5	[X] Pasa[] Fail
VALIDACIÓN			
Resultado Real		Cumple satisfactoriamente las expectativas	
EVIDENCIA			
Sebas Si muy comodo10:21 PM Lau U A mi me pareció especialmente agradable la textura, siento que da un agarre muy sólido10:22 PM			
STATUS GENERAL:		[X] PASÓ [] FALLÓ [] SE BLOQUEÓ [] NO SE EJECUTÓ	

TC-010 Hardware / Térmica Usuario y Seguridad			
Requerimiento	RF-03 / RNF-05 – Temperatura de carcasa $\leq 40^{\circ}\text{C}$ tras 10 min de uso continuo		
Objetivo	Monitorear la temperatura de la carcasa durante 10 min de uso continuo		
Precondiciones	Prototipo ensamblado, termómetro infrarrojo disponible, batería cargada al 100%		
PASOS DEL TEST			
#	INPUT	RESULTADO ESPERADO	STATUS
1	Encender el dispositivo en modo nominal (motor al 60% PWM)	Temperatura inicial de carcasa registrada (valor de referencia)	<i>[X] Pasa[] Fail</i>
2	Registrar temperatura exterior de carcasa cada 2 min durante 10 min	La temperatura no supera los 40°C en ningún punto de la carcasa	<i>[X] Pasa[] Fail</i>
3	Verificar zona de mayor calor (cerca del driver de motor / ESP32)	Temperatura máxima $\leq 40^{\circ}\text{C}$; no hay puntos calientes $> 50^{\circ}\text{C}$ en zonas de contacto con usuario	<i>[X] Pasa[] Fail</i>
VALIDACIÓN			
Resultado Real	Cumple satisfactoriamente con los resultados esperados		
EVIDENCIA			



STATUS GENERAL:	☒ PASÓ ☐ FALLÓ ☐ SE BLOQUEÓ ☐ NO SE EJECUTÓ
-----------------	--

TC-011 Gestión de Energía Usuario y Seguridad			
Requerimiento	RF-04 – Batería debe durar desde 100% hasta corte por bajo voltaje		
Objetivo	Medir el tiempo de descarga de la batería desde el 100% hasta el corte automático		
Precondiciones	Batería cargada al 100%, sistema en modo de operación nominal, cronómetro disponible		
PASOS DEL TEST			
#	INPUT	RESULTADO ESPERADO	STATUS
1	Iniciar operación con batería al 100% en modo nominal (motor 60%, sensor activo, Wi-Fi activo)	Sistema opera normalmente; nivel de batería monitoreable por serial o interfaz	<i>[X] Pasa [] Fail</i>
2	Registrar nivel de batería cada 15 min hasta llegar a zona crítica	Curva de descarga coherente; el sistema emite advertencia al llegar al umbral de bajo voltaje	<i>[] Pasa [X] Fail</i>
3	Esperar el corte automático por protección de bajo voltaje	El sistema se apaga de forma segura antes de que la batería llegue a voltaje de daño (≥ 3.0 V por celda LiPo)	<i>[X] Pasa [] Fail</i>
VALIDACIÓN			
Resultado Real	El modulo de cargador que fue usado para este proyecto tiene protección de alto voltaje, por lo tanto la bateria no se descarga mas de la cuenta		
EVIDENCIA			
No se tiene evidencia fisica del problema			
STATUS GENERAL:	[X] PASÓ [] FALLÓ [] SE BLOQUEÓ [] NO SE EJECUTÓ		

9. Anexo – Diagrama de Estados

